



中山大學
SUN YAT-SEN UNIVERSITY

回顾与补充

补充

1. `printf("%g",x);`
2. 整型极限值
3. 负数表示与八进制
4. 小数转二进制
5. 字符串常量与符号常量
6. 储存类别

补充

1. `printf("%g",x);` 会忽略小数点后的0

7-8 编写程序，键盘输入圆柱体的底面半径和高，计算并输出圆柱体的表面积和体积（圆周率按3.14计）。

输入格式:

在一行中输入圆柱体的底面半径和高。

输出格式:

对每一组输入，在两行中分别输出表面积S和体积V。

输入样例:

在这里给出一组输入。例如：

```
1 1.21
```

输出样例:

在这里给出相应的输出。例如：

```
S=13.8788
```

```
V=3.7994
```

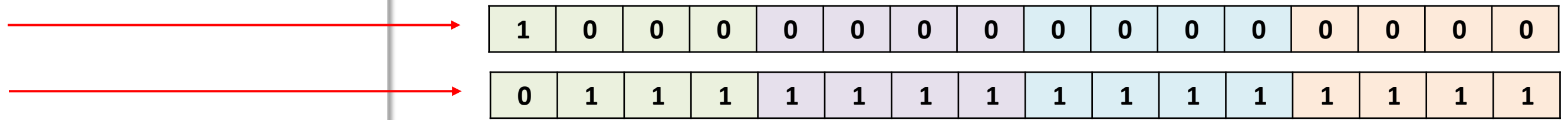
  

补充

2. 整型极限值

```
/*short-max-min*/
#include<stdio.h>
#define UINT_MAX -1
#define SHORT_MAX (unsigned short) UINT_MAX >> 1
#define SHORT_MIN ~SHORT_MAX

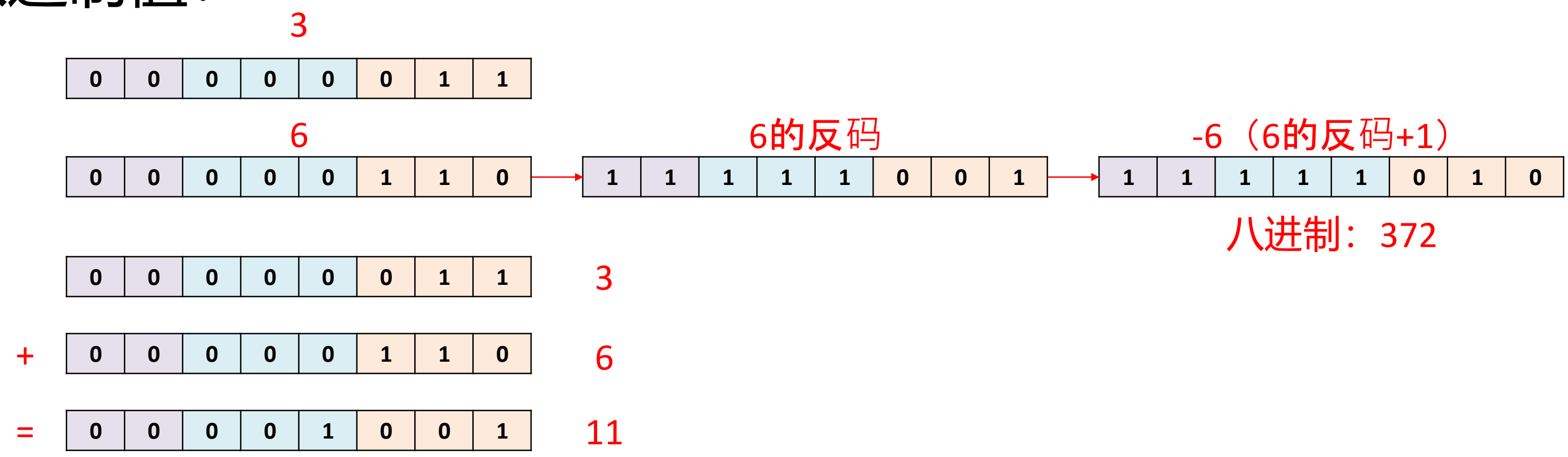
int main(void) {
    short i,j;
    i = 0x8000; // SHORT_MIN;
    j = 0x7FFF; // SHORT_MAX;
    printf("Minmum is %hX , %d\n",i,i);
    printf("Maxmum is %hX , %d\n",j,j);
    return 0;
}
```



补充

3. 以下各变量的八进制值:

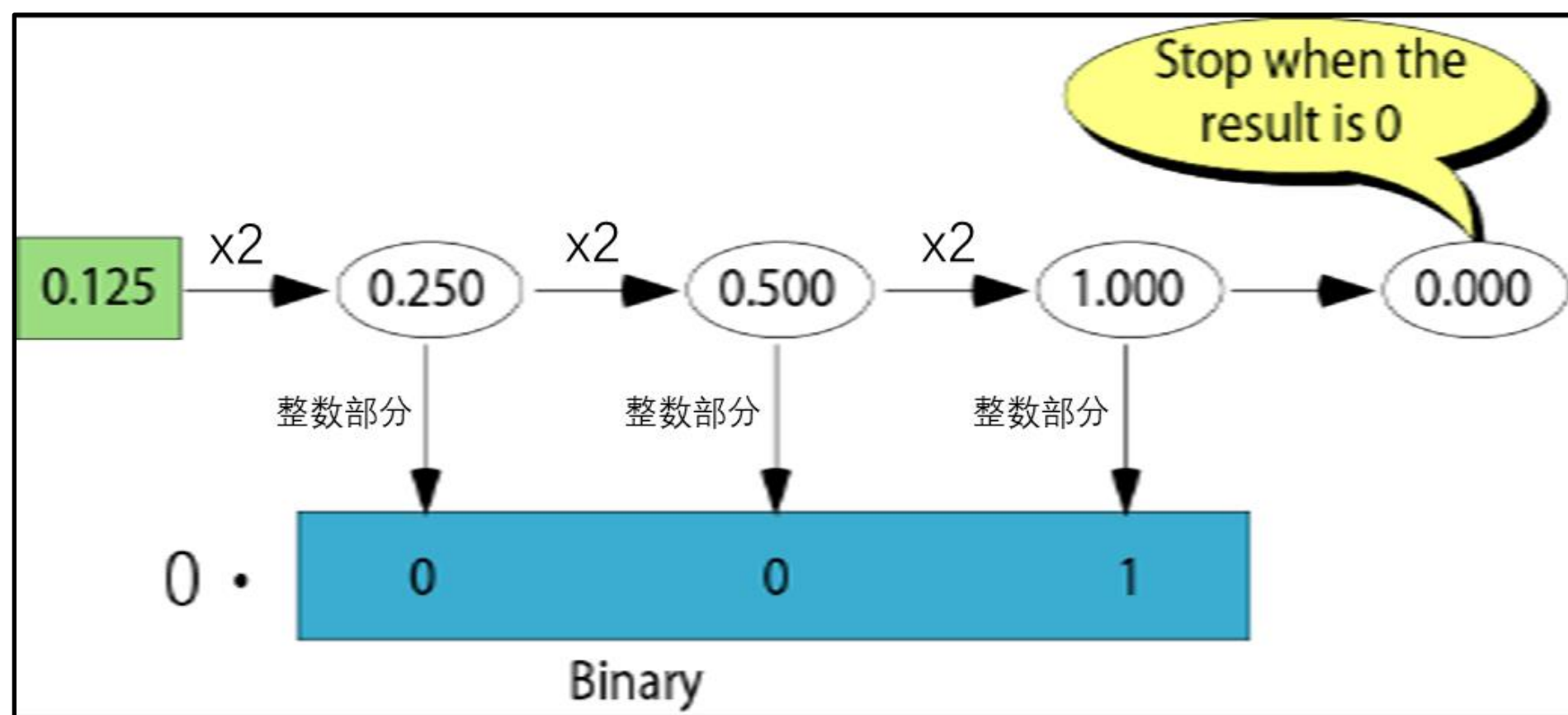
- char x=3;
- char y=-6;
- char z=x-y;



补充

4. 小数转二进制 (IEEE 745标准不要求掌握)

■ 乘2取整



0.15
x2 0.30 -- 0
x2 0.60 -- 0
x2 1.20 -- 1 *
x2 0.40 -- 0 *
x2 0.80 -- 0 *
x2 1.60 -- 1 *
x2 1.20 -- 1 ...

$0.15 = (0.00100110011001...)_{2}$

补充

5. 常量主要包括整数常量、浮点常量、字符常量、**字符串常量与符号常量**

字符串常量

- 含义：连续多个字符组成的字符序列
- 表示：括在双引号中，可以是图形字符，也可以是转义符

```
"abc 123*#"
"\x61 \x62\x63\061 \62\063\x2a\043"
"\r\n\t\\A\\t\x1b\\"1234\xft \x2f\33"
```

- 思考：以下语句会输出什么？

```
printf("\r\n\t\\A\\t\x1b\\"1234\xft \x2f\33\n");
```

符号常量

□ C允许用符号定义常量

```
#define TAXRATE 0.015
```

```
#define PI 3.14159265
```

```
#define TAB '\t'
```

- 编译时替换：程序编译时预处理器会将 TAXRATE 替换为 0.015

□ C90开始允许使用 `const` 关键字限定变量为只读

```
const int MONTHS = 12;
```

字符串 (String) 的定义、输入与输出

□ 示例

```
#include<stdio.h>

int main(void)
{
    char str[233]; // '\0'
    printf("请输入：");
    scanf("%s", str);
    printf("%s\n", str);

    return 0;
}
```

- 读取字符串的时候不需要加&，因为这里str本身就是指针，已经可以认为是地址了

字符串逆序输出

7-5 输入字符串逆序输出

分数 5

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

输入一个字符串，逆序将此字符串按字符一个一个输出

输入格式:

输入字符串，不超过100个字符。

输出格式:

逆序按一个一个字符输出，中间用-隔开。

```
#include <stdio.h>
#include <string.h>

int main(void){
    char str[101];
    scanf("%s", str);

    int len = strlen(str);
    printf("%c", str[len-1]);

    for (int i=len-2; i>=0; i--)
        printf("-%c", str[i]);

    printf("\n");
    return 0;
}
```

补充

6. 储存类别

存储类别

- 存储类别定义了C程序中变量/函数的**可访问性**和**生命周期**，这些说明符放置在它们所修饰的类型之前
- 下面是C中主要用的存储类别
 - 自动变量
 - 寄存器变量
 - 静态变量
 - 外部变量

自动变量

- ❑ 说明符: **auto**
- ❑ auto限定变量只能在函数/块内部使用
- ❑ 自动变量是所有局部变量（函数内部定义的变量）**默认**的存储类，下面两个声明等价：

```
{  
    int x;  
    auto int y;  
}
```

寄存器变量

- ❑ 说明符: **register**
- ❑ register用于“定义”存储在**寄存器**中而不是内存中的局部变量

```
{  
    register int x;  
}
```

- ❑ 意味着变量的最大尺寸等于寄存器的大小，且不能对它应用一元的 **'&'** 运算符（因为它没有内存位置），所以下面这样写是错的

```
{  
    register int x;  
    int *a=&x;  
}
```

静态变量

- ❑ 说明符: **static**
- ❑ 用static修饰变量后, 变量的生命周期变成程序结束为止
- ❑ 用static修饰全局变量, 则全局变量的**作用域**从原来的整个程序, 变为静态全局变量所在的**文件内部**; 修饰局部变量, 作用域不变
- ❑ 用static修饰的变量, 存储位置是**全局数据区** (即静态区)
- ❑ 注: 修饰之前, 局部变量存储在**栈区**

外部变量（全局变量）

- 外部变量的声明在所有函数之外，对所有的程序文件都是可见的
- `extern`说明符提供了一个全局变量的“引用”，当有多个文件且定义了外部变量（或函数）时，可以在其他文件中使用`extern`来得到已定义的变量（或函数）

第一个文件

```
C a.c > ...  
1  int a=1;  
2  
3
```

第二个文件

```
C b.c > ...  
1  #include <stdio.h>  
2  
3  extern int a;  
4  ✓ int main()  
5  {  
6  |   printf("a=%d",a);  
7  }
```

extern存储类

- ❑ 如果把a.c中a的定义加上static就会编译错误，因为用static修饰后对于其他文件就是不可见的了

第一个文件

```
C a.c > ...  
1 static int a=1;  
2  
3
```

编译结果

```
PS C:\Users\86147\Desktop> gcc b.c a.c -o Moon.exe  
C:\Users\86147\AppData\Local\Temp\ccAHgCvX.o:b.c:(.rdata$.refptr.a[.refptr.a]+0x0): undefined reference to `a'  
collect2.exe: error: ld returned 1 exit status
```

中山大学专业课《程序设计I》

第三节 运算符与表达式

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

简单的运算符与表达式

运算符

$+$ $-$ $*$ $/$ $\%$

表达式

$x + y * z$

主要内容

- 1. 算数运算符
- 2. 逻辑运算符
- 3. 关系运算符
- 4. 赋值运算符
- 5. 位运算符
- 6.

Common operators						
assignment	increment decrement	arithmetic	logical	comparison	member access	other
a = b a += b a -= b a *= b a /= b a %= b a &= b a = b a ^= b a <<= b a >>= b	++a --a a++ a--	+a -a a + b a - b a * b a / b a % b ~a a & b a b a ^ b a << b a >> b	!a a && b a b	a == b a != b a < b a > b a <= b a >= b	a[b] *a &a a->b a.b	a(...) a, b (type) a ? : sizeof _Alignof (since C11)



中山大學
SUN YAT-SEN UNIVERSITY

标识符与关键字

Identifiers and Keywords

标识符 (Identifier)

- ❑ 标识符是C语言变量、函数或其他实体的名称
- ❑ 标识符由大小写拉丁字母、数字、下划线组成
- ❑ 哪些标识符是合法的？

age_of_dog	✓	taxRateY2K	PrintHeading	✓	_EAX	✓	Go...od (省略30个o)
Age#	✗	Age-Of-Cat	✗ printHeading	✓	2000TaxRate	✓	✗ 🐱 ✓

❑ 注意：

1. 合法的标识符必须以非数字字符开始
2. 标识符大小写有别
3. C99起允许使用以\uXXXX或\UXXXXXXXX转义记号指定的部分Unicode字符

🐱 = \U0001f431

标识符 (Identifier)

- ❑ 标识符能指代下列类型的实体名称
 - 变量
 - 函数
 - 标签 (struct、union或枚举)
 - 结构体或联合体成员
 - 枚举常量
 - typedef 名
 - 标号名
 - 宏名
 - 宏形参名

常见的标识符命名风格

😞 不推荐的命名方式：

```
PRINTTOPPORTION  
Printtopportion  
Print_Top_Portion  
pRiNtToPpOrTion
```

使用有意义、易读的标识符！

- ❑ **Unix like 风格：** 用大或小写字母，单词间用下划线分割
 - 大写常用于常量，例如：PI, E, INT_MAX
 - 小写常用于局部变量、函数等，例如：printf, x, length, my_age
- ❑ **驼峰风格：** 首字母大写的单词连接一起
 - 大驼峰风格：常用于全局类型名、结构名、函数等，如：Point, ContextMenu
 - 小驼峰风格：常用于局部的变量、函数等，如：textColor, numberOfYears
- ❑ **匈牙利风格：** 用一个前缀表示数据类型
 - 布尔变量isOK，包含名字的字符串strName

C关键词 (Keywords)

- ❑ C关键词：C中保留的标识符 (Reserved Identifiers) 列表
- ❑ 因为语言使用这些关键词，故不可重定义它们

auto	float	signed	_Alignas (C11 起)
break	for	sizeof	_Alignof (C11 起)
case	goto	static	_Atomic (C11 起)
char	if	struct	_Bool (C99 起)
const	inline (C99 起)	switch	_Complex (C99 起)
continue	int	typedef	_Decimal128 (C23 起)
default	long	union	_Decimal32 (C23 起)
do	register	unsigned	_Decimal64 (C23 起)
double	restrict (C99 起)	void	_Generic (C11 起)
else	return	volatile	_Imaginary (C99 起)
enum	short	while	_Noreturn (C11 起)
extern			_Static_assert (C11 起)
			_Thread_local (C11 起)

随着C语言的发展，关键字越来越多

详细参考：
[C 关键词 - cppreference.com](http://cppreference.com)
[标识符 - cppreference.com](http://cppreference.com)

C关键词记忆

❑ C基本数据类型名称

- char short int long float double void
- 建议不要使用的标识符：下划线开头的首字母大写单词，如 `_Bool`；以 `_t` 结尾，如 `size_t`

❑ 对象的修饰、限定或属性名称

- signed unsigned long const static extern volatile auto register

❑ 复杂数据声明

- struct (结构体声明) union (共用体声明) enum (枚举声明) typedef (类型别名声明)

❑ 特殊操作

- sizeof

❑ 程序结构控制

- if else switch case default for while do return continue break goto



中山大學
SUN YAT-SEN UNIVERSITY

表达式 Expressions

表达式 (Expressions)

- ❑ 表达式是运算符 (Operators) 和操作数 (Operands) 的序列, 它指定一个运算
 - 例如: $2 + 3$
 - 其中, “+” 是运算符, “2” 和 “3” 是操作数, “5” 是值
- ❑ C语言中每个表达式都有一个值!

初等表达式与子表达式

□ 表达式

1 + 2 * 3

- “+” 是运算符，操作数 “1” 是**初等表达式** (Primary Expressions)，操作数 “2 * 3” 是**子表达式** (Subexpressions)

□ 初等表达式不能再分解，可能是以下几种

1. 常量及字面量 (例如 2 或 "Hello, world")
2. 适合的已声明标识符 (例如 n 或 printf)
3. 泛型选择 (C11 起)

思考

以下哪条赋值语句是错的？

```
#include <stdio.h>

int main(void) {
    int i,j;
    i = j = 1;    // 赋值运算先算右表达式
    i = (j = 1);  // 赋值运算返回左值
    (i = j) = 1;  // 括号改变了计算顺序

    /*编译提示：(i = j)不是左值表达式*/
}
```

表达式学习要点

- ❑ 值类别 (Value Category)
- ❑ 类型 (Object Type)
- ❑ 求值顺序 (结合性)
- ❑ 运算符优先级 (Precedence)
- ❑ 兼容类型与隐式转换

值类别：左值 (lvalue) 和右值 (rvalue)

- C中每个表达式都有两个独立的属性
 - 表达式求值得到的数据类型，例如：7L 是长整形表达式
 - 表达式属于三个值类别之一：左值、右值（非左值对象）以及函数指代器
- 左值表达式
 1. 左值表达式可作为赋值运算符的左运算数
 2. 左值表达式可以取地址
- 右值表达式（非左值对象）
 - 右值表达式仅能作为赋值运算符的右运算数

左值? 右值?

$x = (x + y) + 5;$



中山大學
SUN YAT-SEN UNIVERSITY

运算符 Operators

从运算到运算符



- **运算**在数学上是一种行为，是通过**已知量**的可能的**组合获得新的量**
- 运算的本质是**集合之间的映射**，而运算符就是反映运算类型的符号
- 例如，算术中的加法 $5 + 3 = 8$ ，这里 5 和 3 是**操作数**，8 是结果，而加号 “+” 是**运算符**表明这是一个加法运算
 - 这是一个常见的二元运算，本质上是 $A \times B \rightarrow C$ 形式的映射
- C语言内置了丰富的运算符，大致将其分为以下六类：**算术运算符、赋值运算符、逻辑运算符、关系运算符、位运算符和其他运算符**

运算符的优先级与结合性

```
/*operator precedence*/
#include <stdio.h>
int main(void)
{
    int a = 16, b = 4, c = 2;

    int d = a + b * c;
    int e = a / b * c;
    printf("d=%d, e=%d\n", d, e);

    return 0;
}
```

乘性运算优先加减运算

二元算术运算左结合

运算符的优先级与结合性

```
/*operator eval order*/
#include <stdio.h>

int main(void) {

    int a=4,b=2,c=3;

    printf("%d\n",a=b=c);
    printf("%d\n", a=b==c);
    printf("%d\n",a==(b=c));
    printf("%d\n", a==(b==c));

    return 0;
}
```

运算符的优先级与结合性

- **优先级**：当多个运算符出现在同一表达式中时，先执行哪个运算符
- **结合性**：当多个优先级相同的运算符共享同一运算对象时，先执行哪个运算符
 - 先执行左边的叫**左结合性**，先执行右边的叫**右结合性**

运算符的优先级与结合性

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= =	从右到左
逗号	,	从左到右

算术运算符 (Arithmetic Operators)

- ❑ 算术运算符应用标准数学运算符到其操作数
- ❑ 操作数类型
 - 字符, 整数, 实数, 复数, 布尔 (C99)
 - [算术类型 - cppreference.com](#)
- ❑ 值类别
 - 非左值, 即不能放在赋值语句左边, 不能求地址等
- ❑ 求值顺序
 - 从左到右结合性 (非一元操作), 即 $a + b + c$ 被分析为 $(a + b) + c$
- ❑ 优先级: 见上一页
- ❑ 隐式转换
 - 两个操作数都转为相同的类型
 - [隐式转换 - cppreference.com](#)

运算符	运算符名	示例	结果
<code>+</code>	一元加	<code>+a</code>	a 提升后的值
<code>-</code>	一元减	<code>-a</code>	a 的相反数
<code>+</code>	加法	<code>a + b</code>	a 加 b
<code>-</code>	减法	<code>a - b</code>	从 a 减去 b
<code>*</code>	乘法	<code>a * b</code>	a 和 b 的积
<code>/</code>	除法	<code>a / b</code>	a 除以 b 的商
<code>%</code>	模	<code>a % b</code>	a 除以 b 的余数
<code>~</code>	逐位非	<code>~a</code>	a 的逐位非
<code>&</code>	逐位与	<code>a & b</code>	a 和 b 的逐位与
<code> </code>	逐位或	<code>a b</code>	a 和 b 的逐位或
<code>^</code>	逐位异或	<code>a ^ b</code>	a 和 b 的逐位异或
<code><<</code>	逐位左移	<code>a << b</code>	a 左移 b 位
<code>>></code>	逐位右移	<code>a >> b</code>	a 右移 b 位

算术运算

位运算

浮点数算术隐式转换 (Implicit Casting)

- ❑ 如果两个操作数类型不一样，则会发生隐式类型转换
- ❑ 类型等级
 - `complex > imaginary > long double > double > float > 整数`
- ❑ 低等级操作数转换到高等级，例如：
 - `1.f + 2000000L;` // long int 转 float; 执行 float 加法
 - `1.0 + 1.0f + 1;` // 左结合, float 转 double, 执行double加法; int 转 double, 执行double 加法
- ❑ float类型变量传入printf()时自动提升为double类型
- ❑ 应用
 - 两个整数i,j需执行浮点除法，表达式写法: `1.0*i/j` 或 `(double)i/j`

整数算术隐式转换 (Implicit Casting)

□ 整数类型等级

■ `long long > long >= int >= short > char`

□ 若高于int等级，则低等级类型操作数转为高等级类型

□ 否则**运算时**操作数自动转为对应的int或unsigned int，即**整型提升**

□ char和short类型变量传入printf()时自动提升为double类型

□ 例如：

■ `a_char + 'a' // a_char转为int`

■ `2 + 3LLU // int 转为 unsigned long long`

■ `2u - 10 // 10 转为 unsigned int, 结果是4294967288 (即 UINT_MAX-7)`

■ `2u - 10LL // 2 从 unsigned 转 long long, 结果是 -8`

整数算术隐式转换 (Implicit Casting)

```
/*ops arithmetic int promotions*/
#include <stdio.h>

int main(void) {
    char i,j;
    /*请分别输入 1, 2, 100, 129*/
    scanf("%hhd",&i);
    j = 255 * i;
    printf("%d, %d, %#x\n",j,255*i,255*i);
}
```

留意变量类型！

隐式类型转换优缺点

- 优点：尽量保证计算结果不溢出
- 缺点：可能产生非预期结果

```
unsigned char x = 250;  
unsigned char y = 10;  
printf("%d\n", x + y);
```

```
unsigned int u = 2;  
int n = -3;  
printf("%u\n", u + n);
```

显式类型转换 (Explicit Casting)

- 从一种类型转变为另一种兼容类型

语法

(类型名) 表达式

其中

类型名 - `void` 类型或任何标量类型

表达式 - 任何标量类型表达式 (除非 类型名是 `void` , 此情)

- 例如:

- `(double)i`

- `(long long)(i+j)`

- 隐式类型转换主要是方便我们书写习惯的算术表达式
- 当需要指定操作数类型时, 则可采用显式转换

[转型运算符 - cppreference.com](http://cppreference.com)

显式类型转换 (Explicit Casting)

```
C a.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 21, b = 4;
6      double c,d;
7      c = (double) a / b;
8      d = (double)(a / b);
9      printf("c : %f\nd : %f\n", c ,d);
10
11 }
```

算术运算符：求商与取余

❑ 假设变量 A 的值为 10，变量 B 的值为 20，则：

运算符	描述	实例
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0

❑ “/” 分为浮点除法和整数除法

- 如果操作数是浮点数，表达式 25.0 / B 得到实数商 1.25； % 运算为非法
- 如果操作数是整数，将会 “趋零取整”
 - 表达式 25 / B 得到商 1； 25 % B 得到余数 5
 - 表达式 -25 / B 得到商 -1； 25 % B 得到余数 -5
 - $a \% b = a - (a / b) * b$

算术运算符： 案例

```
/*ops arithmetic examples*/
#include <stdio.h>

int main(void) {
    int a = 20;
    int b = 10;
    int c ;
    c = a + b;
    printf("Line 1 - c 的值是 %d\n", c );
    c = a - b;
    printf("Line 2 - c 的值是 %d\n", c );
    c = a * b; printf("Line 3 - c 的值是 %d\n", c );
    c = a / b; printf("Line 4 - c 的值是 %d\n", c );
    c = a % b; printf("Line 5 - c 的值是 %d\n", c );
}
```

位运算符

```
/*ops arithmetic quotient*/
#include <stdio.h>

int main(void) {
    int a = 0123456;
    int i = 3;

    printf("Input an octal:");
    scanf("%o",&a);

    /*calculate the i'th number*/
    int q = a >> (3 * (i-1)); // a / 8^(i-1)
    int r = q % 8;

    printf("The %i'th number of the octal %o is %d.",i,a,r);
}
```

位运算符

- 什么是位运算？位运算符作用于位，并逐位执行操作
- $\&$ $|$ $\wedge$ 的真值表如下所示：

p	q	$p \& q$	$p q$	$p \wedge q$
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

位运算： 示例1

□ 假设 $A = 60$, $B = 13$, 现在以二进制格式表示

```
A = 0011 1100
B = 0000 1101
-----
A&B = 0000 1100
A|B = 0011 1101
A^B = 0011 0001
~A  = 1100 0011
```

□ 位运算是对运算数的**每一个二进制位**分别进行

位运算符

□ 假设 $A = 60$ (00111100) , $B = 13$ (00001101) , 则

运算符	描述	实例
&	按位与操作, 按二进制位进行"与"运算。运算规则: $0 \& 0 = 0; 0 \& 1 = 0; 1 \& 0 = 0; 1 \& 1 = 1;$	(A & B) 将得到 12, 即为 0000 1100
	按位或运算符, 按二进制位进行"或"运算。运算规则: $0 0 = 0; 0 1 = 1; 1 0 = 1; 1 1 = 1;$	(A B) 将得到 61, 即为 0011 1101
^	异或运算符, 按二进制位进行"异或"运算。运算规则: $0 \wedge 0 = 0; 0 \wedge 1 = 1; 1 \wedge 0 = 1; 1 \wedge 1 = 0;$	(A ^ B) 将得到 49, 即为 0011 0001
~	取反运算符, 按二进制位进行"取反"运算。运算规则: $\sim 1 = -2; \sim 0 = -1;$	(~A) 将得到 -61, 即为 1100 0011, 一个有符号二进制数的补码形式。
<<	二进制左移运算符, 将一个运算对象的各二进制位全部左移若干位 (左边的二进制位丢弃, 右边补0)	A << 2 将得到 240, 即为 1111 0000
>>	二进制右移运算符, 将一个数的各二进制位全部右移若干位, 正数左补0, 负数左补1, 右边丢弃。	A >> 2 将得到 15, 即为 0000 1111

位运算符-示例2

```
/*ops_bits_examples*/
#include <stdio.h>
int main(void) {
    unsigned int a = 60; /* 60 = 0011 1100 */
    unsigned int b = 13; /* 13 = 0000 1101 */
    int c = 0;
    c = a & b; /* 12 = 0000 1100 */
    printf("Line 1 - c 的值是 %d\n", c );
    c = a | b; /* 61 = 0011 1101 */
    printf("Line 2 - c 的值是 %d\n", c );
    c = a ^ b; /* 49 = 0011 0001 */
    printf("Line 3 - c 的值是 %d\n", c );
    c = ~a; /* -61 = 1100 0011 */
    printf("Line 4 - c 的值是 %d\n", c );
    c = a << 2; /* 240 = 1111 0000 */
    printf("Line 5 - c 的值是 %d\n", c );
    c = a >> 2; /* 15 = 0000 1111 */
    printf("Line 6 - c 的值是 %d\n", c );
}
```

位运算符

- 位运算的优先级：由高到低的顺序是 $\sim \rightarrow \ll、\gg \rightarrow \& \rightarrow \wedge \rightarrow |$
- 位运算的运算对象只能是整数类型的数据

自增、自减运算符

- ❑ 自增/自减运算符是使**对象**的值 **增加/减少 1** 的一元运算符
- ❑ 操作数 (expr)
 - 整型、实数型、指针的**可修改左值**表达式
 - 常见操作数：变量
- ❑ 值类别
 - 非左值，即不能 `&(++i)`
- ❑ 求值顺序
 - 后缀形式：先求值，再修改对象
 - 前缀形式：先修改对象，再求值
- ❑ 优先级（高）
- ❑ **隐式转换（无）**
 - 例如：char i, sizeof(++i)返回1, sizeof(+i)返回4

它们能拥有后缀形式：

`expr ++`

`expr --`

还有前缀形式：

`++ expr`

`-- expr`

++A和A++的区别

```
#include <stdio.h>
int main(void) {
    int c;
    int a = 10;
    c = a++;
    printf("先赋值后运算 : \n");
    printf("Line 1 - c 的值是 %d\n", c );
    printf("Line 2 - a 的值是 %d\n", a );

    a = 10;
    c = a--;
    printf("Line 3 - c 的值是 %d\n", c );
    printf("Line 4 - a 的值是 %d\n", a );
    printf("先运算后赋值 : \n");

    a = 10;
    c = ++a;
    printf("Line 5 - c 的值是 %d\n", c );
    printf("Line 6 - a 的值是 %d\n", a );

    a = 10;
    c = --a;
    printf("Line 7 - c 的值是 %d\n", c );
    printf("Line 8 - a 的值是 %d\n", a );
}
```

思考

□ 以下表达式的值?

1. `int i=1; printf("%d %d\n", i*i++, i);`
2. `j = i++ + i++;`

□ 行为取决于编译器

自增、自减运算符使用原则

1. 不对出现在一个函数的多个参数中的变量使用
2. 不对多次出现在一个表达式中的变量使用

关系运算符

❑ 假设变量 A 的值为 10，变量 B 的值为 20，则

运算符	描述	实例
==	检查两个操作数的值是否相等，如果相等则条件为真	(A == B) 为假
!=	检查两个操作数的值是否相等，如果不相等则条件为真	(A != B) 为真
>	检查左操作数的值是否大于右操作数的值，如果是则条件为真	(A > B) 为假
<	检查左操作数的值是否小于右操作数的值，如果是则条件为真	(A < B) 为真
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真	(A >= B) 为假
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真	(A <= B) 为真

关系运算符： 示例

```
#include <stdio.h>
int main(void) {
    int a = 21;
    int b = 10;
    int c;
    if( a == b ) printf("Line 1 - a 等于 b\n" );
    else printf("Line 1 - a 不等于 b\n" );
    if ( a < b ) printf("Line 2 - a 小于 b\n" );
    else printf("Line 2 - a 不小于 b\n" );
    if ( a > b ) printf("Line 3 - a 大于 b\n" );
    else printf("Line 3 - a 不大于 b\n" );
    /* 改变 a 和 b 的值 */
    a = 5;
    b = 20;
    if ( a <= b ) printf("Line 4 - a 小于或等于 b\n" );
    if ( b >= a ) printf("Line 5 - b 大于或等于 a\n" );
}
```

逻辑运算符

❑ 假设变量 A 的值为 1，变量 B 的值为 0，则：

运算符	描述	实例
&&	如果两个操作数都非零，则条件为真	(A && B) 为假
	如果两个操作数中有任意一个非零，则条件为真	(A B) 为真
!	用来逆转操作数的逻辑状态，如果条件为真则逻辑非运算符将使其为假	!(A && B) 为真

逻辑运算符： 示例

```
#include <stdio.h>
int main(void) {
    int a = 5;
    int b = 20;
    int c;
    if ( a && b ) printf("Line 1 - 条件为真\n" );
    if ( a || b ) printf("Line 2 - 条件为真\n" );
    /* 改变 a 和 b 的值 */
    a = 0;
    b = 10;
    if ( a && b ) printf("Line 3 - 条件为真\n" );
    else printf("Line 3 - 条件为假\n" );
    if ( !(a && b) ) printf("Line 4 - 条件为真\n" );
}
```

在C语言当中，所有 **非0** 的数都会是**真**也就是1，而 **0** 会是**假**也就是仍然是0

交换两个数

问题：如何交换两个数？

交换两个数

```
#include <stdio.h>
int main(void) {
    double a,b,c;

    scanf("%lf%lf",&a,&b);

    c=a;
    a=b;
    b=c;

    printf("a=%lf,b=%lf\n",a,b);

    return 0;
}
```

交换两个数

问题：有没有其他办法？

交换两个数

```
#include <stdio.h>
int main(void) {
    double a,b;

    scanf("%lf%lf",&a,&b);

    a = a + b;
    b = a - b;
    a = a - b;

    printf("a=%lf,b=%lf\n",a,b);

    return 0;
}
```

变量	a	b
初始值	x	y
a=a+b	x+y	y
b=a-b	x+y	x
a=a-b	y	x

这么做可能有什么问题？ 若为整数可能会溢出 😅

交换两个数

```
#include <stdio.h>
int main() {
    int a,b,c;

    scanf("%d,%d",&a,&b);

    a = a ^ b;
    b = b ^ a;
    a = a ^ b;

    printf("a=%d,b=%d\n",a,b);

    return 0;
}
```

$x \wedge x = 0$
 $x \wedge 0 = x$
 $x \wedge y \wedge y = x$

变量	a	b
初始值	x	y
a=a^b	x^y	y
b=a^b	x^y	x^(y^y)
a=a^b	y	x

这么做可能有什么问题？ 若a和b相同.....😅

运算优化

```
int i,j;  
i = 257 / 8;  
j = 456 % 32
```

运算优化

```
int i,j;  
i = 257 / 8;  
j = 456 % 32
```

```
int i,j;  
i = 257 >> 3;  
j = 456 - (456 >> 4 << 4);
```

这么做可能有什么问题？ 换个数试试.....😂

位运算应用：掩码 (Mask)

A = 1010 1010

B = 0000 1111 (掩码)

A&B = 0000 1010 屏蔽(清零)高4位

A|B = 1010 1111 置位低4位

A^B = 1010 0101 高4位直通, 低4位求反

按位与的实际用途：将一个数的某位清零

- 如果想将某个数的某个比特清零，只要另找另一个数，让对应的该比特位置0，然后使两数进行按位与&运算
- 例如：有数为00101011，想让该数从右数的第4比特变成0，其他位保持不变。另找一个数，该数从右数的第4比特设它为0，其他位全为1：11110111，将两个数进行&运算：

$$\begin{array}{r} 00101011 \\ \& 11110111 \\ \hline 00100011 \end{array}$$

按位与的实际用途：取一个数中某些指定位

- 如有一个两个字节整数 $a=(16a1)_{16}$ ，想取出其中的低字节
- 构造数 b ，它的高字节全为0，低字节全为1，即 $b=(00ff)_{16}=(255)_{10}$ ，只需将 a 与 b 按位与即可：

$$\begin{aligned} a &= (16a1)_{16} = (0001011010100001)_2 \\ \& b &= (00ff)_{16} = (\underline{0000000011111111})_2 \\ (00a1)_{16} &= (0000000010100001)_2 \end{aligned}$$

异或的实际用途：使特定位翻转

- ❑ 假设有01111010，想使其低4位翻转，即1变为0，0变为1。可以将它与00001111进行 $\wedge$ 运算，即

$$\begin{array}{r} 0\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \wedge\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1 \\ \hline 0\ 1\ 1\ 1\ 0\ 1\ 0\ 1 \end{array}$$

- ❑ 要使哪几位翻转就将其与进行 $\wedge$ 运算的该几位置为1即可

取反的实际用途

- 若一个16位整数a，想使最低一位为0，可以将a与二进制数1111111111111110 (0177776)₈进行按位与：

```
00000000000111101
^ 1111111111111110
-----
00000000000111100
```

- 但若将此程序移植到32位计算机上，由于整数用4个字节表示，想将最后一位变成0就**不能用**a&0177776了，应**改用**a&0377777777776
- 这个算法的移植性很差，可以改用： a=a&~1
- 在以2字节存一个整数时，1的二进制形式为0000000000000001，~1是1111111111111110，以4个字节存储一个整数时，~1是1111111111111111111111111110

赋值运算符

运算符	描述	实例
=	简单的赋值运算符，把右边操作数的值赋给左边操作数	$C = A + B$ 将把 $A + B$ 的值赋给 C
+=	加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数	$C += A$ 相当于 $C = C + A$
-=	减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数	$C -= A$ 相当于 $C = C - A$
*=	乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数	$C *= A$ 相当于 $C = C * A$
/=	除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数	$C /= A$ 相当于 $C = C / A$
%=	求模且赋值运算符，求两个操作数的模赋值给左边操作数	$C \% = A$ 相当于 $C = C \% A$
<<=	左移且赋值运算符	$C << = 2$ 等同于 $C = C << 2$
>>=	右移且赋值运算符	$C >> = 2$ 等同于 $C = C >> 2$
&=	按位与且赋值运算符	$C \& = 2$ 等同于 $C = C \& 2$
^=	按位异或且赋值运算符	$C \wedge = 2$ 等同于 $C = C \wedge 2$
=	按位或且赋值运算符	$C = 2$ 等同于 $C = C 2$

赋值运算符： 示例

```
#include <stdio.h>
int main() {
    int a = 21;
    int c ;
    c = a;
    printf("Line 1 - = 运算符实例, c 的值 = %d\n", c );
    c += a;
    printf("Line 2 - += 运算符实例, c 的值 = %d\n", c );
    c -= a;
    printf("Line 3 - -= 运算符实例, c 的值 = %d\n", c );
    c *= a;
    printf("Line 4 - *= 运算符实例, c 的值 = %d\n", c );
    c /= a;
    printf("Line 5 - /= 运算符实例, c 的值 = %d\n", c );
    c = 200;
    c %= a;
    printf("Line 6 - %= 运算符实例, c 的值 = %d\n", c );
    c <<= 2;
    printf("Line 7 - <<= 运算符实例, c 的值 = %d\n", c );
    c >>= 2;
    printf("Line 8 - >>= 运算符实例, c 的值 = %d\n", c );
    c &= 2;
    printf("Line 9 - &= 运算符实例, c 的值 = %d\n", c );
    c ^= 2;
    printf("Line 10 - ^= 运算符实例, c 的值 = %d\n", c );
    c |= 2;
    printf("Line 11 - |= 运算符实例, c 的值 = %d\n", c );
}
```

其他运算符

运算符	描述	实例
sizeof()	返回变量的大小	sizeof(a) 将返回 4, 其中 a 是整数
&	返回变量的地址	&a; 将给出变量的实际地址
*	指向一个变量	*a; 将指向一个变量
? :	条件表达式	如果条件为真 ? 则值为 X : 否则值为 Y

其他运算符： 示例

```
#include <stdio.h>
int main() {
    int a = 4;
    short b;
    double c;
    int* ptr;
    /* sizeof 运算符实例 */
    printf("Line 1 - 变量 a 的大小 = %lu\n", sizeof(a) );
    printf("Line 2 - 变量 b 的大小 = %lu\n", sizeof(b) );
    printf("Line 3 - 变量 c 的大小 = %lu\n", sizeof(c) );
    /* & 和 * 运算符实例 */
    ptr = &a; /* 'ptr' 现在包含 'a' 的地址 */
    printf("a 的值是 %d\n", a);
    printf("*ptr 是 %d\n", *ptr);
    /* 三元运算符实例 */
    a = 10;
    b = (a == 1) ? 20: 30;
    printf("b 的值是 %d\n", b );
    b = (a == 10) ? 20: 30;
    printf("b 的值是 %d\n", b );
}
```

运算符优先级

- 运算符的优先级确定表达式中项的组合，这会影响到一个表达式如何计算，某些运算符比其他运算符有更高的优先级
 - 例如，乘除运算符具有比加减运算符更高的优先级
 - 如： $x = 7 + 3 * 2$ ，在这里， x 被赋值为 13，而不是 20，因为运算符 $*$ 具有比 $+$ 更高的优先级，所以首先计算乘法 $3*2$ ，然后再加上 7
- 下表将按运算符优先级从高到低列出各个运算符，具有较高优先级的运算符出现在表格的上面，具有较低优先级的运算符出现在表格的下面，在表达式中，较高优先级的运算符会优先被计算

运算符优先级

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= =	从右到左
逗号	,	从左到右

运算符优先级： 示例

```
#include <stdio.h>
int main() {
    int a = 20;
    int b = 10;
    int c = 15;
    int d = 5;
    int e;
    e = (a + b) * c / d; // ( 30 * 15 ) / 5
    printf("(a + b) * c / d 的值是 %d\n", e );
    e = ((a + b) * c) / d; // (30 * 15) / 5
    printf("((a + b) * c) / d 的值是 %d\n", e );
    e = (a + b) * (c / d); // (30) * (15/5)
    printf("(a + b) * (c / d) 的值是 %d\n", e );
    e = a + (b * c) / d; // 20 + (150/5)
    printf("a + (b * c) / d 的值是 %d\n", e );
    return 0;
}
```

谢谢大家！

邮箱: yuesh5@mail.sysu.edu.cn